

Lab 1: Reasoning in FOL

Lab session activities

Francisco Lobo and Joseph Razavi*

Academic session: 2024-25

Learning objectives

At the end of this laboratory exercise you should be able to:

1. Model problems using first-order logic (FOL).
2. Translate first-order formulas into the TPTP format.
3. Run Vampire on TPTP problems and explain its output; extracting proofs and models.
4. Relate Vampire options to the underlying refinements of the given-clause algorithm.

The first two relate to material you've been practising since the start of the course, the others to material on FOL reasoning that we'll develop from Week 4. There will be 2 lab sessions for this reasoning lab and in the first one you should complete the lab activities that follow; they introduce content to get you ready to work on the subsequent lab exercises.

Preliminaries

The course materials will cover the general theory of automated theorem proving. In this lab exercise we will focus on the **Vampire theorem prover**. This is installed on the Ubuntu image of the Kilburn lab machines

```
$ /opt/vampire/vampire --version
Vampire 4.9 (commit 5ad494e78 on 2024-06-14 14:05:27 +0100)
Linked with Z3 4.8.12.0
```

Note: In case you'd like to get Vampire installed on your own machine, you can build it from source by following instructions at <https://github.com/vprover/vampire>; there are also some pre-compiled Linux binaries available for download. You can use any relatively modern version of Vampire to complete the lab activities, although you'll need a version with Z3 support to explore proofs that involve arithmetic. However, we only support the version of Vampire on Kilburn machines shown above, so you must ensure that all your code works in the lab machines before submission!

You are expected to gain experience using Vampire by engaging with the formative activities of this lab manual, which are highlighted with a  icon. Hopefully the manual is reasonably self contained, but if you want more information on Vampire here are some sources to look into:

- Giles Reger gave a day tutorial on Vampire in 2017 and the material can be found [on GitHub](#). The [theory slides](#) give a good introduction to the given-clause algorithm that Vampire uses.
- The unofficial technical manual for Vampire is the [CAV 2013 tutorial paper](#). It gives a good theoretical overview of Vampire, how it works, and some of its options.

Vampire has lots of features that we won't explore in this lab and its default settings aren't necessarily aligned with the classical automated theorem proving algorithms that the course explores. For this reason, we provide a wrapper shell script called `run_vampire` with the lab materials, which

*Many of the activities for this lab were developed by Giles Reger.

sets sensible options to complete the formative activities. You should get familiar with the presets that this Vampire wrapper script offers.

```
$ ./run_vampire --help
Usage: run_vampire [--preset gc|discount|otter] [--verbose off|gc|all] [options...] [problem]
Runs Vampire with sensible default/preset options on [problem].

Sets global options that
- disable Vampire command warnings for safe options
- fix TPTP for input syntax
Sets preprocessing options that
- enable equality proxy transformation (-ep RSTC)
- disable function and predicate definition removal (-fde none -updr off)
- disable introduction of named subformulas (-nm 0)
Sets proof search options that
- fix literal selection for unordered resolution (-s 0)
- disable AVATAR clause splitting during saturation (-av off)

The 3 possible --preset options are
- gc|given_clause: mimics the plain given clause loop
- discount: mimics the discount loop
- otter: mimics the otter loop
The gc|given_clause preset options are used by default.
These presets have different simplification options
- gc|given_clause does not apply backward or forward subsumption
  (-bs off -bsr off -fs off -fsr off)
- discount and otter: apply backward and forward subsumption
  (-bs on -bsr on -fs on -fsr on)

The 3 possible --verbose options are
- off: normal vampire output
- gc: show information to help follow the given clause loop
- all: show (almost) all debug information

You can override each of these settings using [options...] arguments.
Use /opt/vampire/vampire --help to get further information on Vampire.
```

Figure 1: The run_vampire wrapper

Note: If you install Vampire on your own machine, then you'll probably need to edit the run_vampire wrapper and fix the location of the Vampire binary by changing /opt/vampire/vampire appropriately.

In order to explore theorem proving with Vampire, you'll need to adjust the run_vampire presets, override some of its settings, experiment with additional Vampire options, etc. To do this you just need to give extra command-line arguments as indicated above. You can access extensive information about Vampire from the command-line

```
$ ./run_vampire --show_options on
```

Reading this documentation will help you understand the behaviour of Vampire. If you're interested in what a specific option does then you can run, for example

```
$ ./run_vampire -explain ep
```

🚶 You should read Vampire's help for the various options that run_vampire sets. In the case of the ep option, it reports that this controls the proxy predicate that can be used to treat *equality*. You have seen how to introduce an equality proxy in the notes.

Recall that in first-order logic there is no assumption of unique names, and so equality isn't the same as syntactic "identity". Instead, equality in FOL is understood as a binary predicate with some well-defined axioms, which restricts the possible models for the particular theory. If you turn the equality proxy off, then you'll find that Vampire use advanced features like *superposition* or *demodulation*.

Important: You will need to execute Vampire via the `run_vampire` wrapper in order to reproduce the results shown in the lab manual. However, this wrapper will not be available during marking! As part of your solution to lab exercises you will need to specify **all** of the Vampire command-line arguments necessary for it to run on your code.

Modelling problems with TPTP

You should aim to complete the activities in this part by the end of your first lab session. The goal is to practise the first two learning objectives on a toy example.

Using FOL to describe knowledge about a particular situation is one of the core objectives of this course unit. As an example, suppose that in some world:

- (a) We define being happy as loving someone.
- (b) We assume rich people love money.
- (c) We know that Giles is rich.

It is clear that both *happy* and *rich* should be (unary) predicates about people. We have information about a specific person, which we can denote by a constant, say *giles*. Intuitively, we distinguish loving another person from the love of money. But in the ignorance of this semantic distinction, we could use a single binary predicate *loves* to syntactically encode these two kinds of love. We then represent the above knowledge in FOL as

- (a) $\forall x. (\text{happy}(x) \leftrightarrow \exists y. (\text{loves}(x, y)))$
- (b) $\forall x. (\text{rich}(x) \rightarrow \text{loves}(x, \text{money}))$
- (c) $\text{rich}(\text{giles})$

Given this knowledge we could ask if Giles is happy in this world. Mathematically, this is the same as checking if the axioms (a) to (c) entail the following conjecture

- (d) $\text{happy}(\text{giles})$

To solve this problem using an automated theorem prover like Vampire, the first step is to write it down in a format that can be processed. We'll use the syntax of [The TPTP Problem Library](#), which is almost exactly an ASCII version of the mathematical notation. You can learn all about TPTP by reading the [description of the language](#) and the [manual of the problem library](#). But you should start by working through the slides of the [TPTP World Tutorial](#), in particular the examples for [propositional logic](#) and for [first-order logic](#).

✚ In the TPTP language the above problem can be written as follows.

```
fof(a, definition, ![X]:( happy(X) <=> (?[Y]: loves(X,Y)) )).
fof(b, axiom, ![X]:( rich(X) => loves(X,money) )).
fof(c, axiom, rich(giles)).
fof(d, conjecture, happy(giles)).
```

You should save this on a file called `greed.p` to use in the next activity. This knowledge-base consists of four declarations, each of which is terminated by a full stop. All of these declarations have the structure *form(name,role,formula)*.

The **form** describes the logic being used.

We will usually use **fof** standing for *first-order form*, **cnf** standing for *clausal normal form*, but you will also encounter **fff** standing for *typed first-order form*.

The **name** is an identifier given to the formula.

Vampire will only use this in proofs with the option `--output_axiom_names on`. These identifiers do not impact proof search at all! Names don't even need to be distinct.

The **role** describes the kind of formula.

We will mainly use **definition**, **axiom** and **conjecture**, but you will also see others like **type** being used. The meaning of a TPTP problem file is that the conjunction of the definition and axiom formulas entails the conjecture formula. There can be at most one conjecture. It

is possible to have no conjecture, in which case we are just checking the consistency of the knowledge-base.

The **formula** is the logical formula being described.

The syntax is what one might expect for an ASCII based representation of FOL formulas. It is important to note that in TPTP formulas

- words that begin with upper-case letters are *variables*
- words that begin with lower-case letters are *constant*, *function*, or *predicate* symbols

You have seen the TPTP notation in the notes, but briefly the relevant logical syntax is

- $\sim$ for not
- $\&$ for conjunction (and), $|$ for disjunction (or)
- $\Rightarrow$ for implication, $\Leftrightarrow$ for equivalence
- $![X]$: for universal quantification over X
- $?[X]$: for existential quantification over X
- $=$ for equality and $\neq$ for disequality

Note that you can group quantifiers of the same kind. Thus $![X,Y]$ can replace $![X] : ![Y]$:

✚ To practise your understanding of TPTP, try writing out the following formula in this syntax

$$\forall x. (P(f(x)) \rightarrow (\exists y. Q(x, y)))$$

You need to think about what is the signature of this FOL language, and how to represent it in TPTP. If that was tricky, go back to and read this section again or ask for help. Now put your code in a file called `syntax.p` and then check whether it is syntactically correct by running

```
$ ./run_vampire --mode output syntax.p
```

This will print your TPTP problem back to you if you got the syntax right, or give you a (hopefully helpful) error message if you didn't.

✚ The first thing that Vampire does when solving a problem is to put it into clausal form. You have studied how to do this by hand in the lectures, but you can make Vampire do it for you by running the following command.

```
$ ./run_vampire --mode clausify syntax.p
cnf(u5,axiom,
  q(X0,sK0(X0)) | ~p(f(X0))).
```

As you can see, the output of Vampire is itself in TPTP format. Does the result make sense to you? Can you write out the resulting clause in the first-order logic syntax we are used to? Now do the same with the `greed.p` file.

Note: When doing the lab exercises, it's a good idea to always check the clausal normal form that Vampire generates. This is an easy way to, for example, verify that you haven't missed any set of parenthesis.

Proving things with Vampire

You should aim to complete the activities in this part by the end of your assigned lab session. The goal is to practise the third learning objective on a toy example.

The `greed.p` problem we introduced in the previous section ask us to establish whether

$$\left\{ \begin{array}{l} \forall x. (\text{happy}(x) \leftrightarrow \exists y. (\text{loves}(x, y))) \\ \forall x. (\text{rich}(x) \rightarrow \text{loves}(x, \text{money})) \\ \text{rich}(\text{giles}) \end{array} \right\} \models \text{happy}(\text{giles})$$

The technique that Vampire uses for this is called *proof by refutation*, that is based on the result of Proposition 2 in the notes. Instead of asking if the set of premises (on the left) entails the conjecture (on the right), we negate the conjecture and try to show that the resulting set of formulas

$$\left\{ \begin{array}{l} \forall x. (\text{happy}(x) \leftrightarrow \exists y. (\text{loves}(x, y))) \\ \forall x. (\text{rich}(x) \rightarrow \text{loves}(x, \text{money})) \\ \text{rich}(\text{giles}) \\ \neg \text{happy}(\text{giles}) \end{array} \right\}$$

is *unsatisfiable*, which means that it has no models. It turns out that this is something that can be done proof theoretically, because from a set of unsatisfiable formulas we must be able to derive **false**. It follows that to construct a proof simply we need to work out all the consequences of a given set of formulas, with respect to a (complete) inference system for FOL, and check that one of these is **false**. The process of extending a set of formulas¹ with its consequences is known as *saturation*, and when doing proof by refutation we can stop this process as soon as we find **false**.

The main derivation rule of Vampire's inference system is called *resolution*. This is a generalisation of *modus ponens*, which you have studied for propositional logic in COMP11120. We'll be covering *unification* of FOL formulas in the lectures, which is a necessary technique to use modus ponens in first-order logic. As an example, given formulas (b) and (c) in the `greed.p` problem we can derive

$$\frac{\text{rich}(x) \rightarrow \text{loves}(x, \text{money}) \quad \text{rich}(\text{giles})}{\text{loves}(\text{giles}, \text{money})} \quad [x \mapsto \text{giles}]$$

because $[x \mapsto \text{giles}]$ is the *most general unifier (mgu)* for the formulas $\text{rich}(x)$ and $\text{rich}(\text{giles})$. Under this substitution, the RHS of the implication is the derived formula $\text{loves}(\text{giles}, \text{money})$.

The formal definition of the resolution rule is

$$\frac{\neg A \vee F \quad B \vee G}{(F \vee G)\theta} \quad \theta = \text{mgu}(A, B)$$

where A, B are atoms and F, G arbitrary clauses.

Important: Despite writing clauses with logical syntax, we always treat clauses as sets of literals when doing resolution! Thus it is possible for F or G to be *empty clauses* in the above rule. For example, if $G = \{\}$ is the empty set of literals then the premise on the RHS simplifies because $\{B\} \cup G = \{B\}$. In FOL syntax this could be written as $B \vee \text{false} \equiv B$.

- ✂ Look again at the above modus ponens example using `greed.p` formulas, and convince yourself that it is an application of the formal resolution rule. You should find that in this case $A = \text{rich}(x)$, $B = \text{rich}(\text{giles})$, $F = \text{loves}(x, \text{money})$, and G is empty. Once you gain experience with the resolution calculus, you'll get comfortable writing clauses in FOL syntax, reading them as sets of literals, and identifying the empty clause with **false**.

Using Vampire to solve the `greed.p` problem we get the output shown in Figure 2. It reports a **Theorem** status because a refutation was found. The output first shows the full clausification process and then the reasoning steps. The clausification process details when a choice of Skolem function is made, etc. It includes several transformations Vampire performs which were unnecessary for this problem, like the intermediate *flattening* rewrite. The proof then consists of **resolution** reasoning steps, the first of which corresponds to the above modus ponens example.

The proof ends when Vampire finds the empty clause. If we trace the steps backwards, discarding no-op steps, we obtain a tree of the Vampire run. This is shown in Figure 3. The leaves of this tree are the three premises (1, 2, and 15), the negated conjecture (16), and the choice of Skolem function (10).

¹Conceptually the negated conjecture is no different from any of the other formulas in the set, even if theorem provers like Vampire will treat conjectures differently so the proof search is goal oriented.

```

$ ./run_vampire greed.p -s 1 -awr 1:1
% Refutation found. Thanks to Tanya!
% SZS status Theorem for greed
% SZS output start Proof for greed
5 1. ! [X0] : (happy(X0) <=> ? [X1] : loves(X0,X1)) [input]
2. ! [X0] : (rich(X0) => loves(X0,money)) [input]
3. rich(giles) [input]
4. happy(giles) [input]
5. ~happy(giles) [negated conjecture 4]
10 6. ~happy(giles) [flattening 5]
7. ! [X0] : (loves(X0,money) | ~rich(X0)) [ennf transformation 2]
8. ! [X0] : ((happy(X0) | ! [X1] : ~loves(X0,X1)) & (? [X1] : loves(X0,X1) | ~happy(X0))) [nnf transformation 1]
9. ! [X0] : ((happy(X0) | ! [X1] : ~loves(X0,X1)) & (? [X2] : loves(X0,X2) | ~happy(X0))) [rectify 8]
10. ! [X0] : (? [X2] : loves(X0,X2) => loves(X0,sk0(X0))) [choice axiom]
15 11. ! [X0] : ((happy(X0) | ! [X1] : ~loves(X0,X1)) & (loves(X0,sk0(X0)) | ~happy(X0))) [skolemisation 9,10]
13. ~loves(X0,X1) | happy(X0) [cnf transformation 11]
14. ~rich(X0) | loves(X0,money) [cnf transformation 7]
15. rich(giles) [cnf transformation 3]
16. ~happy(giles) [cnf transformation 6]
20 17. loves(giles,money) [resolution 14,15]
18. happy(giles) [resolution 17,13]
19. $false [resolution 18,16]
% SZS output end Proof for greed
% -----

```

Figure 2: Solving the `greed.p` problem

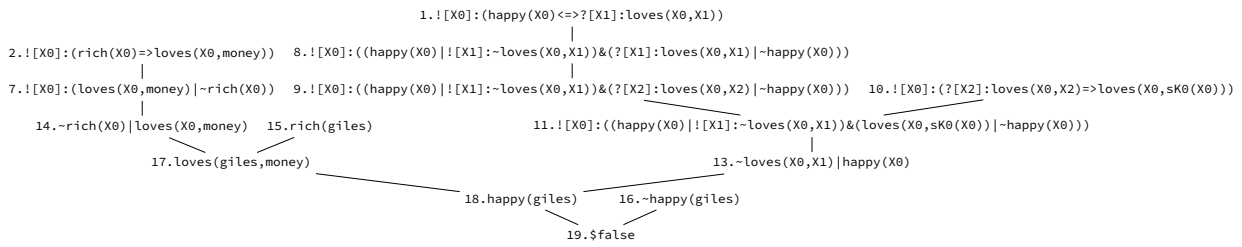


Figure 3: A run of Vampire on the `greed.p` problem

The pre-processing and clausification stages of the run simply prepare the problem for Vampire's algorithm. These do not form part of the proof by refutation, which only involves the resulting set of clauses. The actual logical proof is shown in Figure 4.

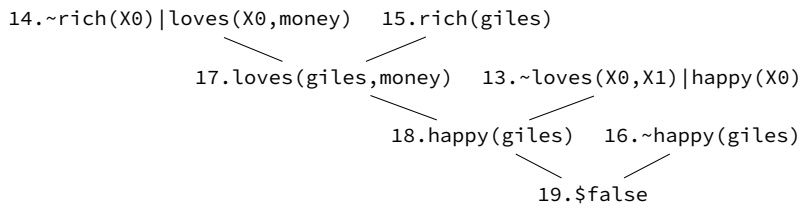


Figure 4: Solving `greed.p` with `-s 1 -awr 1:1`

Your GitLab repo has a Python script called `spec2latex.py` that will help you generate trees for the proofs you develop in this lab. It reads a JSON specification of the proof that defines:

- An object `vampire_run` with the command-line arguments necessary to obtain the proof using Vampire.
- An object `formulas` with a dictionary mapping TPTP formulas to proof tree node numbers.
- An array `proof` that recursively specifies the tree (as a Lisp-style list of lists) of number nodes making the refutation.
- An object `resolution` with full details of a resolution step. This will be explained in the next section.

By default the script outputs the LaTeX code that creates the proof tree, but it can optionally generate the PDF directly if you have a working installation of [TeX Live](#).

```

{
  "vampire_run": {
    "-ep": "RSTC",
    "-fde": "none",
    "-nm": 0,
    "-updr": "off",
    "-s": 1,
    "-awr": "1:1",
    "-av": "off",
    "-sa": "discount",
    "-bs": "off",
    "-bsr": "off",
    "-fs": "off",
    "-fsr": "off",
    "--bad_option": "off",
    "--input_syntax": "tptp"
  },
  "formulas": {
    "loves(X0,sK0(X0))|~happy(X0)": 12,
    "~loves(X0,X1)|happy(X0)": 13,
    "~rich(X0)|loves(X0,money)": 14,
    "rich(giles)": 15,
    "~happy(giles)": 16,
    "loves(giles,money)": 17,
    "happy(giles)": 18,
    "$false": 19,
    "loves(giles,sK0(giles))": 20
  },
  "proof": [
    19,
    [18,
      [17, [14], [15]],
      [13]
    ],
    [16]
  ],
  "resolution": {
    "active": [16, 15, 13, 14],
    "passive": [12],
    "unprocessed": [17],
    "premises": [14, 15],
    "conclusion": 17,
    "mgu": [
      {"X0": "giles"},
      {}
    ]
  }
}

```

Figure 5: A proof specification for the `greed.p` problem

✚ The tree in Figure 4 was made using the JSON specification in Figure 5 by running the commands

```

$ ./spec2latex.py greed.json
$ pdflatex greed.tex

```

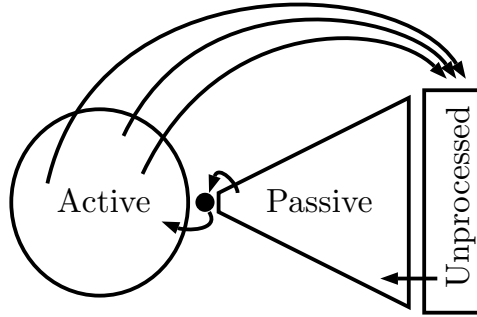
Note that in the proof specification `vampire_run` includes **all** the options that the `run_vampire` wrapper sets, as well as the options `-s 1 -awr 1:1` that were specified on the command-line.

To answer some of the lab exercises you will need to make JSON specifications (like that of Figure 5) for proofs that you construct using Vampire. Your GitLab repo includes a file called `template.json` which you can adapt to make specifications for your proofs.

Understanding Vampire's algorithm

You should aim to complete the activities in this part by the end of your assigned lab session. The goal is to practise the fourth learning objective on a toy example.

Vampire runs a *given-clause saturation algorithm* that we'll discuss in the lectures. It works by iteratively selecting a clause from a set of *passive* clauses, adding it to a set of *active* clauses, and performing all inferences between the selected clause and the active set, which are temporarily cached as an *unprocessed* set of clauses. The following diagram illustrates the flow of clauses from the currently unselected passive set to the currently selected active set, and the caching of unprocessed inferences.



We can ask Vampire to show the saturation process to see the order in which clauses are selected and which clauses are produced by activating (selecting) them. The option `--show_active` on prints out the clauses Vampire selects for activation. The option `--show_new` on prints out all of the clauses produced by inferences. The option `--proof_extra free` includes additional information about clauses in the output. These 3 options are triggered using the preset `--verbose gc` of the `run_vampire` wrapper. Solving the `greed.p` problem using this we get the output shown in Figure 6 before the refutation proof of Figure 2.

```

$ ./run_vampire greed.p -s 1 -awr 1:1 --verbose gc
[SA] new: 13. happy(X0) | ~loves(X0,X1) [cnf transformation 11] {a:0,w:5,thAx:0,allAx:2,thDist:-2}
[SA] new: 12. loves(X0,sk0(X0)) | ~happy(X0) [cnf transformation 11] {a:0,w:6,thAx:0,allAx:2,thDist:-2}
[SA] new: 14. loves(X0,money) | ~rich(X0) [cnf transformation 7] {a:0,w:5,thAx:0,allAx:1,thDist:-1}
5 [SA] new: 15. rich(giles) [cnf transformation 3] {a:0,w:2,thAx:0,allAx:1,thDist:-1}
[SA] new: 16. ~happy(giles) [cnf transformation 6] {a:0,w:2,goal:1,thAx:0,allAx:1,thDist:-1}
[SA] active: 16. ~happy(giles) [cnf transformation 6] {a:0,w:2,nSel:1,goal:1,thAx:0,allAx:1,thDist:-1}
[SA] active: 15. rich(giles) [cnf transformation 3] {a:0,w:2,nSel:1,thAx:0,allAx:1,thDist:-1}
[SA] active: 13. ~loves(X0,X1) | happy(X0) [cnf transformation 11] {a:0,w:5,nSel:1,thAx:0,allAx:2,thDist:-2}
10 [SA] active: 14. ~rich(X0) | loves(X0,money) [cnf transformation 7] {a:0,w:5,nSel:1,thAx:0,allAx:1,thDist:-1}
[SA] new: 17. loves(giles,money) [resolution 14,15] {a:1,w:3,thAx:0,allAx:2,thDist:-2}
[SA] active: 17. loves(giles,money) [resolution 14,15] {a:1,w:3,nSel:1,thAx:0,allAx:2,thDist:-2}
[SA] new: 18. happy(giles) [resolution 17,13] {a:2,w:2,thAx:0,allAx:4,thDist:-4}
[SA] active: 12. ~happy(X0) | loves(X0,sk0(X0)) [cnf transformation 11] {a:0,w:6,nSel:1,thAx:0,allAx:2,thDist:-2}
15 [SA] active: 18. happy(giles) [resolution 17,13] {a:2,w:2,nSel:1,thAx:0,allAx:4,thDist:-4}
[SA] new: 19. $false [resolution 18,16] {a:3,w:0,goal:1,thAx:0,allAx:5,thDist:-5}
[SA] new: 20. loves(giles,sk0(giles)) [resolution 18,12] {a:3,w:4,thAx:0,allAx:6,thDist:-6}
% Refutation found. Thanks to Tanya!

```

Figure 6: Saturation for the `greed.p` problem

The saturation process that Vampire completed to solve `greed.p` is shown in lines 5 to 20 of Figure 6. The first 6 lines report the order in which the **cnf** clauses for the problem passed from the unprocessed to the passive set. After this Vampire starts activating clauses, but no inferences are possible until formula 14 becomes active and it derives the clause `loves(giles,money)` just as described in the modus ponens example of the previous section. This resolution step involves the premises (14 and 15), the most general unifier $[x \mapsto \text{giles}]$, and the conclusion (17). The state of the given-clause saturation process at this point is given by the list of passive (only 12 is unselected), active (16, 15, 13, and 14 were selected), and unprocessed clauses (17 was the only clause generated by the selection of 14).

All this information is collected in the `resolution` object of the JSON specification in Figure 5. Note that the representation of the mgu must be a pair of JSON objects: the first fixes a map for variables in the LHS premise, and the second a map for variables in the RHS premise. In this example, the LHS formula 14 has one free variable `x0` and the RHS formula 15 has no free variables at all. You may find situations in which resolution premises have free variables in common that their most general unifier will map differently.

- ✚ Now continue following the given-clause algorithm for yourself. Could you give the full details of the resolution step which obtains formula 18? How about the one that derive the empty clause 19? You will need to construct these when answering some of the lab exercises.

Important: Vampire has various options to show additional information about its state. However, remember that Vampire’s algorithm includes many refinements that go beyond the basic given-clause algorithm explained above. In the lab exercises you will be asked to track the state of this basic given-clause saturation process (for the clause selection approach that you configure using Vampire’s options). In general, this will be different from the one Vampire reports when run with the `--show_everything` on option for example.

To help you understand Vampire’s output better, we’ll briefly explain the full meaning of line 15 in Figure 6. The part `[SA] active:` means that this is coming from the Saturation Algorithm and is about clause activation (when a clause is selected in the given clause algorithm). The 17 is the id number of the clause — clauses are numbered as they are produced. Then we have the clause `loves(giles,money)` in clausal form. The next bit `[resolution 14,15]` describes how the clause was generated, here it was by resolution from clauses 14 and 15. Finally, there is the extra information string `{a:1,w:3,nSel:1,thAx:0,allAx:2,thDist:-2}` giving values that are used for clause and literal selection, etc.

The important fields for understanding Vampire’s saturation strategy are:

- the age of the clause `a`,
- the weight of the clause `w`,
- the number of selected literals in the clause `nSel`
(which are always the leftmost literals in active clauses), and
- the `goal` if the clause is derived from the negated conjecture.

Using the command line option `-s` we can control how Vampire does literal selection. The default value of `run_vampire` corresponds to *unordered* resolution, which allows all literals to be chosen during resolution. If we use `-s 1` we get *ordered* resolution, which means only maximal literals are chosen as atoms for resolution. You can control how Vampire performs clause selection using the command line option `-awr` which controls the ratio between selecting clauses by age and by weight. For example, if you specify `-awr 10:1` then for every 10 clauses selected by age Vampire will select one by weight.

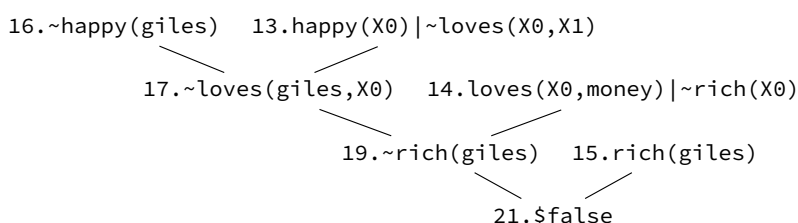


Figure 7: Solving `greed.p` with `-s 0 -awr 1:10`

Figures 7 and 8 show how changing clause and literal selection strategies results in different proof trees for the `greed.p` problem. Indeed, each generates the empty clause by resolving on a different literal: `happy(giles)` in Figure 4, `rich(giles)` in Figure 7, and `loves(giles,money)` in Figure 8. This last clause, which we have been tracking since the previous section, isn’t even present in the second tree.

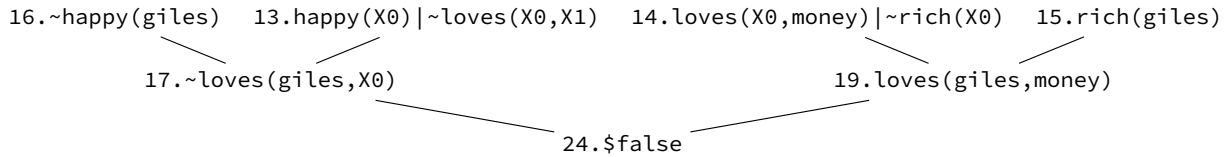


Figure 8: Solving `greed.p` with `-s 0 -awr 10:1`

In the lectures we will study different clause and literal selection strategies. In this lab you'll be asked to **experiment** with these and explore the results. You need to record the various `-s` and `-awr` values that you use in the `vampire_run` object of your proof specifications.

- ✚ In preparation for the lab exercises make sure that you are able to reproduce the proof trees in Figures 7 and 8 before the end of your lab session. In your JSON specifications, add the details for a resolution that only appears in that particular tree, including a complete description of the saturation process state, as appears in Figure 5. This will allow you to better understand why we arrive at different proofs.

Formative Exercise

Consider the following problem in FOL **with** equality.

$$\left\{ \begin{array}{l} \forall x. (\text{require}(x) \rightarrow \text{require}(\text{depend}(x))) \\ \text{depend}(\text{clang}) = \text{llvm} \\ \text{depend}(\text{llvm}) = \text{libc} \\ \text{require}(\text{clang}) \end{array} \right\} \models \text{require}(\text{libc})$$

In this activity you'll use Vampire to build different proofs for this problem. Two proofs are considered different **if and only if** each involves a resolution step which doesn't appear in the other. Remember that every resolution step consists of 2 premises, 1 mgu, and 1 conclusion.

Translate the problem to TPTP and save your code as `deps.p` then use Vampire to obtain 3 proofs which are **mutually different** and satisfy the following criteria.

- Not all of your proofs can have the same equality proxy setting.
- Not all of your proofs can have the same selection setting.
- Not all premises of your resolution steps are ground clauses.

You should save your proofs as `deps-a.json`, `deps-b.json`, and `deps-c.json` in the proof spec format. Each one of your proofs needs to include the details of a resolution step and corresponding state of the saturation algorithm, which shows that the proof is different from the other two. These must be supported by a corresponding map of formulas to indices, and specifications also have to include the Vampire options used to generate the proof.